

Nexus Lexis - Frontend Integration Guide (OTP Skipped)

NL-FE-INT-002 | v2.0 | 4 August 2026

1. Purpose

This guide covers production API integration for the Nexus Lexis frontend without signup OTP. Email OTP is temporarily disabled while Microsoft resolves external mail delivery (550 5.7.708). When OTP is re-enabled, use Appendix A to add the verification step.

Backend repo: <https://github.com/habibafaisal15-cpu/NexusLexisBackend>

2. Production environment variables

Create `.env.production` in the frontend:

```
VITE_API_BASE_URL=https://nexus-lexis-backend-ql8w.vercel.app/api/v2
VITE_AUTH_API_URL=https://nexus-lexis-backend-45v4.vercel.app/api/auth
VITE_LEX_API_BASE_URL=https://nexus-lexis-backend-ql8w.vercel.app/api/v1/lex
```

Variable | Value | Purpose

Do **not** use WebSocket for LEX in production. Vercel does not support WebSockets.

3. Service URLs and health checks

Service | Base URL | Health

Check OTP mode (important):

```
GET https://nexus-lexis-backend-45v4.vercel.app/api/auth/config
```

Expected while OTP is skipped:

```
{
  "signupOtpRequired": false,
  "registerRoles": ["client", "lawyer", "ca"],
  "authMethods": ["email", "google"]
}
```

Or check health:

```
GET https://nexus-lexis-backend-45v4.vercel.app/api/health
-> "signupOtpSkipped": true
```

4. Authentication overview

Signup (no OTP): POST /register -> tokens + user -> complete profile

Login: POST /login -> accessToken + refreshToken -> dashboard

Google (alternative): POST /google/token -> tokens + user

All protected requests:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

5. Signup flow (OTP skipped)

Step 1 - Validate email (optional but recommended)

POST {VITE_AUTH_API_URL}/register/validate

Content-Type: application/json

```
{ "email": "user@example.com" }
```

Response (available):

```
{ "valid": true, "available": true }
```

Response (taken):

```
{ "valid": true, "available": false, "code": "ALREADY_EXISTS", "error": "..." }
```

Step 2 - Register directly (no OTP, no verificationToken)

POST {VITE_AUTH_API_URL}/register

Content-Type: application/json

```
{
  "fullName": "John Doe",
  "email": "user@example.com",
  "password": "securepass123",
  "phone": "+923001234567",
  "role": "client"
}
```

Field | Required | Notes

fullName

Success response (201):

```
{
  "accessToken": "<JWT>",
  "refreshToken": "<opaque-token>",
  "token": "<JWT>",
  "expiresIn": "24h",
  "refreshExpiresAt": "...",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "fullName": "John Doe",
    "role": "client",
    "roles": ["client"]
  }
}
```

Store accessToken and refreshToken. Use accessToken for all API calls.

Step 3 - Complete profile (after register)

Client:

PUT {VITE_AUTH_API_URL}/profile/client

Authorization: Bearer <accessToken>

```
{
  "cnic": "12345-1234567-1",
  "address": "Street, City",
}
```

```
"city": "Karachi",
"phone": "+923001234567"
}
```

Lawyer application:

```
POST {VITE_AUTH_API_URL}/profile/lawyer/apply
Authorization: Bearer <accessToken>
(multipart or JSON - see profile fields in API reference)
```

CA application:

```
POST {VITE_AUTH_API_URL}/profile/ca/apply
Authorization: Bearer <accessToken>
```

6. Login

```
POST {VITE_AUTH_API_URL}/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "securepass123"
}
```

Response: Same shape as register (accessToken, refreshToken, user).

Errors:

Message | Action

`Invalid

7. Token refresh and logout

Refresh (when API returns 401):

```
POST {VITE_AUTH_API_URL}/refresh
Content-Type: application/json

{ "refreshToken": "<stored-refresh-token>" }
```

Returns new accessToken and new refreshToken (rotation - replace stored token).

Logout:

```
POST {VITE_AUTH_API_URL}/logout
Content-Type: application/json

{ "refreshToken": "<stored-refresh-token>" }
```

8. Google Sign-In

Option A - Google Identity Services (recommended)

```
POST {VITE_AUTH_API_URL}/google/token
Content-Type: application/json

{
  "idToken": "<credential from Google GIS>",
  "role": "client"
}
```

Use credential field if your GIS library returns that name instead of idToken.

Option B - Redirect flow

```
GET {VITE_AUTH_API_URL}/google/url
-> { "url": "https://accounts.google.com/..." }
```

Redirect user to url. After consent, Google redirects to Auth callback, then user lands on:

```
{FRONTEND_URL}/login?token=<accessToken>&refreshToken=<refreshToken>
```

Parse tokens from query string and store them.

9. Current user

```
GET {VITE_AUTH_API_URL}/me
Authorization: Bearer <accessToken>
```

Response:

```
{
  "user": {
    "id": 1,
    "email": "...",
    "fullName": "...",
    "role": "client",
    "roles": ["client"],
    "dashboardUserId": 1,
    "phone": "",
    "profile": { ... }
  }
}
```

Full profile bundle:

```
GET {VITE_AUTH_API_URL}/profile
Authorization: Bearer <accessToken>
```

Switch dashboard role (users with multiple roles):

```
POST {VITE_AUTH_API_URL}/profile/switch-role
Authorization: Bearer <accessToken>
```

```
{ "role": "lawyer" }
```

Returns updated JWT with new active role.

10. Main API (dashboard)

Base: VITE_API_BASE_URL with Authorization: Bearer <accessToken>.

Client workspace

```
GET /workspace
GET /notifications
GET /documents
GET /orders
POST /orders
GET /library/catalog
POST /appointments
GET /appointments
GET /subscription
GET /invoices
POST /evaluations
GET /lawyers/public
```

Lawyer routes (prefix `/lawyer`)

```
GET /lawyer/dashboard
GET /lawyer/profile
PATCH /lawyer/profile
GET /lawyer/clients
GET /lawyer/appointments
GET /lawyer/orders
```

CA routes (prefix `/ca`)

```
GET /ca/dashboard
GET /ca/profile
GET /ca/taxation/profiles
GET /ca/clients
```

Messages

```
GET /messages
POST /messages
```

Some requests may need:

```
X-Client-Role: CorporateClient | LegalAdvocate | CharteredAccountant
```

11. LEX AI chat

```
POST {VITE_LEX_API_BASE_URL}/chat/
Content-Type: application/json
```

```
{
  "message": "How do I register a company in Pakistan?"
}
```

Response:

```
{
  "response": "...",
  "language": "EN",
  "register": "PLAIN",
  "show_lawyer": false
}
```

Behaviour | Example

Intro /

Sessions (optional):

```
GET {VITE_LEX_API_BASE_URL}/sessions/
GET {VITE_LEX_API_BASE_URL}/sessions/{sessionKey}/
```

12. Demo accounts (QA)

Role | Email | Password

Client

Use for testing dashboards before real signup is wired.

13. Frontend signup UI (recommended)

While `signupOtpRequired === false`:

1. Sign-up form - fullName, email, password, role, optional phone
2. Submit -> POST /register (skip OTP screens entirely)
3. On success -> store tokens -> redirect to profile completion or dashboard
4. Google button -> POST /google/token as alternative

Hide or disable OTP input fields until backend sets `signupOtpRequired: true`.

14. Common errors

Error | Cause | Fix

15. CORS and deployment checklist

Step | Owner | Action

Appendix A - Signup OTP (enable later)

When backend sets `SKIP_SIGNUP_OTP=false` and `signupOtpRequired: true`:

```
Step 1: POST /register/send-otp    -> { "email": "user@example.com" }
Step 2: POST /register/verify-otp -> { "email": "...", "code": "123456" }
      -> returns { "verificationToken": "..." }
Step 3: POST /register            -> { ..., "verificationToken": "..." }
```

Check GET /api/auth/config on app load - if `signupOtpRequired` is true, show OTP screens.

Appendix B - Forgot password (OTP still applies)

Password reset still uses email OTP (when mail delivery works):

```
POST /forgot-password    -> { "email" }
POST /forgot-password/verify-otp -> { "email", "code" }
POST /reset-password     -> { "email", "resetToken", "password" }
```

Google-only accounts cannot reset password - show "Continue with Google".

Document end - Nexus Lexis Frontend Integration (OTP Skipped v2.0)